Lesson:

Number System

PW SKILLS

Topics

- What is a number system?
- Types of number system.
- Conversion of binary to decimal.
- Conversion of decimal to binary.

Introduction

Numbers are a fundamental concept in mathematics and computing. In JavaScript, you encounter various number systems, and understanding them is crucial for effective programming. This guide provides an overview of number systems, specifically focusing on binary and decimal representations, along with conversion examples.

What is a number system?

A number system, also known as a numeral system, is a mathematical notation for expressing numbers. It defines how we represent and manipulate numbers using symbols and rules. The two most commonly encountered number systems are the decimal system and the binary system.

Types of number system

- **Decimal (Base 10):**

The decimal number system is the most common number system in everyday use. It uses ten digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, and 9.

- **Binary (Base 2):**

The binary number system uses only two digits: 0 and 1. It is commonly used in computing and digital electronics.

- **Octal (Base 8):**

The octal number system uses eight digits: 0, 1, 2, 3, 4, 5, 6, and 7. It's less commonly used today but can still be found in some computer systems.

- **Hexadecimal (Base 16):**

The hexadecimal number system uses sixteen digits: 0-9 and A-F (or a-f). It's often used in computing for representing binary values more compactly.

- **Ternary (Base 3):**

The ternary number system uses three digits: 0, 1, and 2. It's used in some mathematical and computer science contexts.

- **Quaternary (Base 4):**

The quaternary number system uses four digits: 0, 1, 2, and 3. It's less common than binary, octal, or hexadecimal.

Each of these number systems has its own set of rules and applications. The choice of a number system depends on the specific use case and the requirements of a given problem or application.

Conversion of Binary to Decimal

Binary and decimal are two different number systems used in computing. Binary uses only two digits (0 and 1), while decimal uses ten digits (0-9). Converting binary to decimal is a fundamental skill in computer science and programming.

To convert a binary number to decimal, you follow these steps:

1. **Write Down the Binary Number:** Start with the binary number you want to convert.
2. **Assign Powers of 2:** Assign powers of 2 to each bit in the binary number, starting with 2^0 for the rightmost bit and increasing by 1 as you move to the left.
3. **Calculate the Values:** Calculate the value of each term by multiplying the bit value (0 or 1) by 2 raised to the assigned power.
4. **Sum the Values:** Add up all the values you calculated in step 3. The sum is the decimal equivalent of the binary number.

Now, let's look at some examples.

Example 1: Convert 1101 (Binary) to Decimal

1. Start with the binary number 1101.
2. Assign powers of 2 from right to left: 2^0, 2^1, 2^2, and 2^3.
3. Calculate the values: 1 * 2^3 + 1 * 2^2 + 0 * 2^1 + 1 * 2^0.
4. Sum the values: 8 + 4 + 0 + 1 = 13.

So, 1101 in binary is equal to 13 in decimal.

Example 2: Convert 10101 (Binary) to Decimal

1. Start with the binary number 10101.
2. Assign powers of 2 from right to left: 2^0, 2^1, 2^2, 2^3, and 2^4.
3. Calculate the values: 1 * 2^4 + 0 * 2^3 + 1 * 2^2 + 0 * 2^1 + 1 * 2^0.
4. Sum the values: 16 + 0 + 4 + 0 + 1 = 21.

So, 10101 in binary is equal to 21 in decimal.

Example 3: Convert 11100 (Binary) to Decimal

1. Start with the binary number 11100.
2. Assign powers of 2 from right to left: 2^0, 2^1, 2^2, 2^3, and 2^4.
3. Calculate the values: 1 * 2^4 + 1 * 2^3 + 1 * 2^2 + 0 * 2^1 + 0 * 2^0.
4. Sum the values: 16 + 8 + 4 + 0 + 0 = 28.

So, 11100 in binary is equal to 28 in decimal

Conversion of decimal to binary

Converting decimal numbers to binary is a fundamental skill in computer science and programming. The binary number system uses only two digits (0 and 1) to represent numbers, which is different from the decimal system that uses ten digits (0-9). Understanding how to convert between these two systems is important in digital computing.

To convert a decimal number to binary, you can use a method known as "repeated division by 2." Here's how it works:

1. **Start with the Decimal Number:** Begin with the decimal number you want to convert.
2. **Divide by 2:** Divide the decimal number by 2.
3. **Record the Remainder:** Write down the remainder (0 or 1) that you get when you divide by 2.
4. **Repeat the Process:** Continue dividing the quotient from the previous step by 2, and record the remainders.
5. **Read the Remainders Backwards:** To get the binary representation, read the remainders from the last division to the first. This is your binary number.

Now, let's look at some examples.

Example 1: Convert 26 (Decimal) to Binary

1. Start with the decimal number 26.
2. Divide by 2: Quotient = 13, Remainder = 0.
3. Divide by 2: Quotient = 6, Remainder = 1.
4. Divide by 2: Quotient = 3, Remainder = 0.
5. Divide by 2: Quotient = 1, Remainder = 1.
6. Divide by 2: Quotient = 0, Remainder = 1.
7. Read the remainders from the bottom up: 11010.

So, 26 in decimal is equal to 11010 in binary.

Example 2: Convert 14 (Decimal) to Binary.

1. Start with the decimal number 14.
2. Divide by 2: Quotient = 7, Remainder = 0.
3. Divide by 2: Quotient = 3, Remainder = 1.
4. Divide by 2: Quotient = 1, Remainder = 1.
5. Divide by 2: Quotient = 0, Remainder = 1.
6. Read the remainders from the bottom up: 1110.

So, 14 in decimal is equal to 1110 in binary.

Example 3: Convert 37 (Decimal) to Binary.

1. Start with the decimal number 37.
2. Divide by 2: Quotient = 18, Remainder = 1.
3. Divide by 2: Quotient = 9, Remainder = 1.
4. Divide by 2: Quotient = 4, Remainder = 0.
5. Divide by 2: Quotient = 2, Remainder = 0.
6. Divide by 2: Quotient = 1, Remainder = 1.
7. Divide by 2: Quotient = 0, Remainder = 1.
8. Read the remainders from the bottom up: 100101.

So, 37 in decimal is equal to 100101 in binary.

How to Convert Binary to Decimal and Vise-versa in JavaScript

Now let's see the conversion of binary to decimal and conversion of decimal to binary in JS using **parseInt**.

parseInt is a built-in JavaScript function that is used to parse a string and convert it into an integer (a whole number). It is often used for converting string representations of numbers, including binary, octal, decimal, or hexadecimal, into their numeric equivalents.

The basic syntax of parseInt is as follows:

Syntax

```
parseInt(string, radix);
```

Here's an example of binary-to-decimal conversion and decimal-to-binary conversion using parseInt:

Binary to Decimal Conversion (JavaScript Example)

Ex:

```
// Convert binary (base 2) to decimal (base 10)
const binaryValue = "1101";
const decimalEquivalent = parseInt(binaryValue, 2);
console.log(`Binary: ${binaryValue} => Decimal:
${decimalEquivalent}`);  // Binary: 1101 => Decimal: 13;
```

In this example:

1. We have a binary value, "1101".
2. We use parseInt(binaryValue, 2) to convert it to a decimal value. The 2 specifies that the input is in base 2 (binary).

Decimal to Binary Conversion (JavaScript Example)

Ex:

```
// Convert decimal (base 10) to binary (base 2)
const decimalValue = 26;
const binaryEquivalent = parseInt(decimalValue,
10).toString(2);
console.log(`Decimal: ${decimalValue} => Binary:
${binaryEquivalent}`);  //Decimal: 26 => Binary: 11010;
```

In this example:

1. We have a decimal value, 26.
2. We use parseInt(decimalValue, 10) to ensure that JavaScript interprets it as a decimal value (base 10).
3. Then, we use .toString(2) to convert the decimal value to a binary string representation.